

PROJECT DOCUMENTATION

API Automation Framework

REST API Test Automation using Java, Rest Assured & JUnit 5

GitHub Link: “[RajaVUst/SDET-3 at Saiteja Final Assessment](#)”

Field	Details
Project Name	API Automation Framework
Prepared By	Saiteja Kodi
Date	July 21, 2026

1. Project Overview

This project is a REST API test automation framework built to validate the functionality, reliability, and data integrity of API endpoints. It follows a clean, reusable structure using Java, Rest Assured, and JUnit 5, and is integrated with a CI/CD pipeline for automated execution.

Objective

To design and implement a maintainable API automation framework that validates status codes, response data, and JSON schemas, with clear reporting and continuous integration support.

2. Technology Stack

Technology	Purpose
Java 21	Core programming language
Maven	Build and dependency management
Rest Assured	API request/response automation
JUnit 5	Test execution framework
Jackson	JSON serialization / deserialization
Allure Report	Test execution reporting
Log4j2	Logging framework
GitHub Actions	CI/CD pipeline
JSON Schema Validation	Response structure validation

3. Project Structure

```
api-automation-framework/
|-- src/
|   |-- main/java/
|   |   |-- base/           -> Base test setup, request/response specs
|   |   |-- config/        -> Environment configuration (.env loader)
|   |   |-- models/        -> POJO classes (e.g. User model)
|   |   |-- `-- utils/     -> Common reusable helper methods
|   |-- `-- test/java/
|       |-- `-- tests/     -> API test classes (JUnit 5)
|-- src/test/resources/
|   |-- schemas/           -> JSON Schema files for validation
|   |-- `-- log4j2.xml     -> Logging configuration
|-- .github/workflows/
|   |-- `-- ci.yml         -> GitHub Actions CI pipeline
|-- .env                  -> Local environment variables
|-- `-- pom.xml            -> Maven project configuration
```

Folder	Description
base/	Contains BaseTest and Request/Response specification setup
config/	Loads environment values from .env and GitHub Secrets
models/	POJO classes used for request and response mapping
utils/	Reusable helper and utility methods
tests/	API test classes covering the implemented scenarios
schemas/	JSON Schema files used for response validation
workflows/	GitHub Actions CI pipeline definition

4. Work Implemented

- Environment configuration managed using a .env file locally and GitHub Secrets in CI
- Base Test setup to initialize Rest Assured configuration before test execution
- Request and Response Specifications defined for consistent API calls
- Reusable API framework with common methods for GET, POST, and other calls
- User model (POJO) created for request and response mapping
- API test classes implemented to cover the defined test scenario
- Logging implemented using Log4j2 for tracking request/response flow
- Allure Report integrated for clear, visual test execution reporting
- JSON Schema Validation added to verify response structure correctness
- GitHub Actions CI pipeline configured to run tests automatically

5. Test Scenario

The framework implements the following end-to-end API flow:

- Create User — Send a request to create a new user
- Generate Token — Authenticate and generate an access token for the created user
- Retrieve Books — Call the books endpoint using the generated token
- Validate Status Codes — Confirm each API call returns the expected HTTP status
- Validate Response Body — Verify actual response values against expected data
- Validate JSON Schema — Confirm the response structure matches the defined schema

6. Validations Performed

Validation Type	Description
Status Code Validation	Confirms each endpoint returns the correct HTTP status code
Response Validation	Verifies response fields match expected values
Schema Validation	Validates response structure against a JSON Schema
Assertions	JUnit 5 assertions used to verify expected outcomes
Authentication Validation	Confirms token generation and authorized access to endpoints

7. Commands Used

```
# Build the project
mvn clean install

# Run tests
mvn test

# Generate Allure report
allure generate allure-results --clean

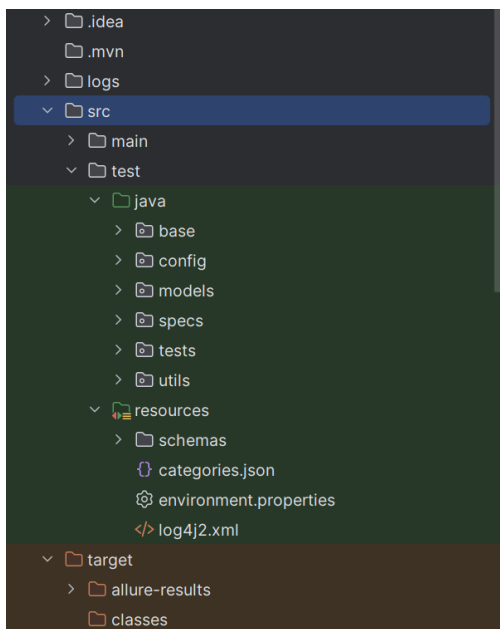
# Open Allure report
allure open allure-report
```

8. CI/CD

The GitHub Actions workflow automatically checks out the code, sets up Java and Maven, installs dependencies, runs the test suite, and generates the Allure report on every push or pull request. This ensures the API tests are validated continuously without manual effort.

9. Screenshots

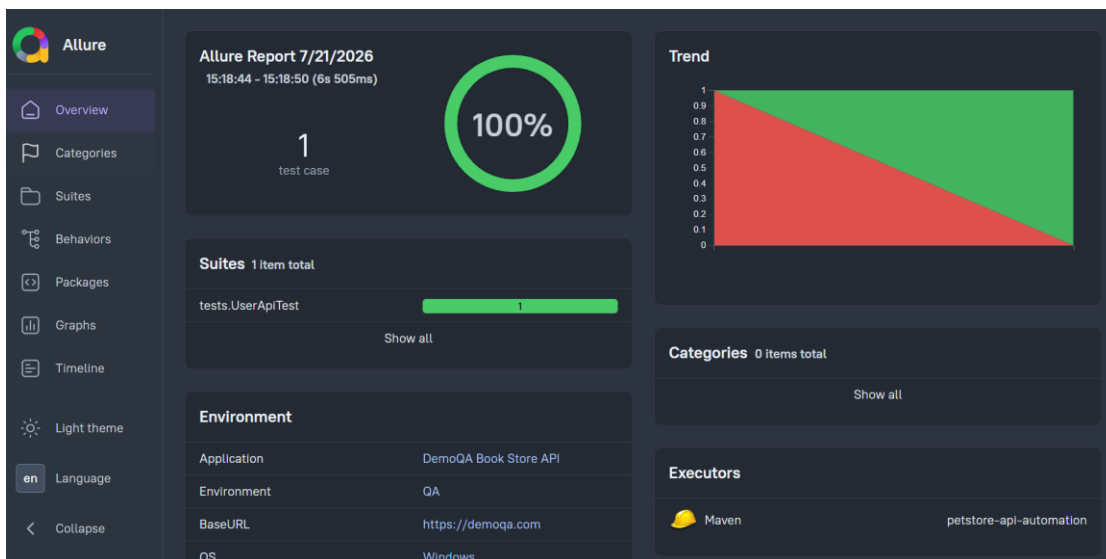
Structure Screenshot:



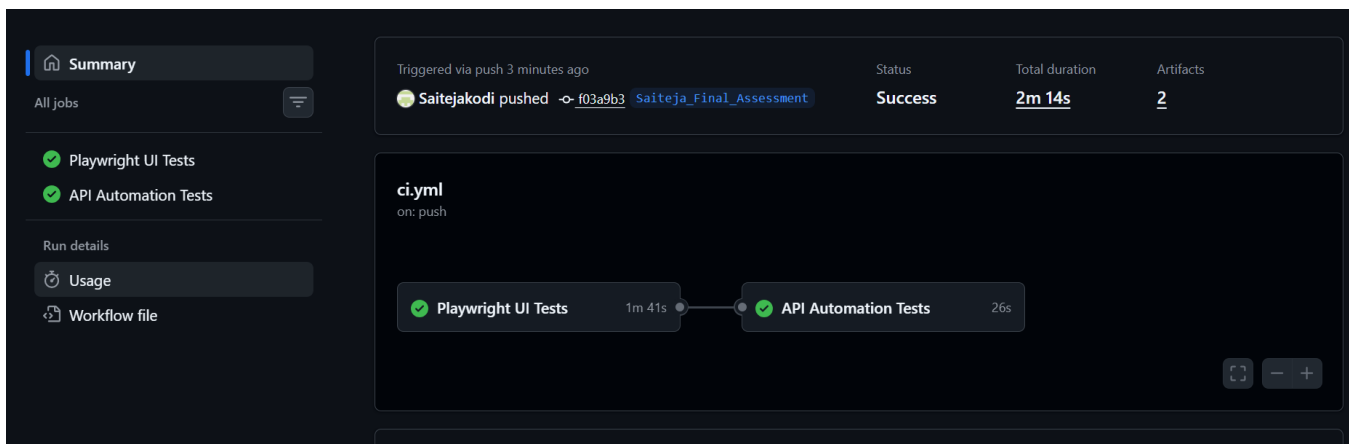
Test Execution and Console Output:

```
Terminal Local x + v
[INFO] Running tests.UserApiTest
2026-07-21 15:18:44 INFO [main] UserApiTest - Creating user...
2026-07-21 15:18:47 INFO [main] UserApiTest - User created successfully.
2026-07-21 15:18:47 INFO [main] UserApiTest - Generating token...
2026-07-21 15:18:50 INFO [main] UserApiTest - Token generated successfully.
2026-07-21 15:18:50 INFO [main] UserApiTest - Fetching books...
2026-07-21 15:18:50 INFO [main] UserApiTest - Books fetched successfully.
2026-07-21 15:18:50 INFO [main] UserApiTest - API flow completed successfully.
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 7.020 s -- in tests.UserApiTest
[INFO] Results:
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 11.585 s
[INFO] Finished at: 2026-07-21T15:18:51+05:30
[INFO]
```

Allure Dashboard:



GitHub Actions Successful Run:



10. Challenges Faced

Challenge	Resolution
Managing sensitive data across environments	Used .env locally and GitHub Secrets in CI to keep credentials secure
Token dependency between test steps	Generated the token in a setup step and reused it across dependent tests
Validating dynamic response structures	Used JSON Schema Validation instead of hardcoded field checks
Readability of console logs	Configured Log4j2 for structured and readable logging

11. Conclusion

The API Automation Framework successfully validates the end-to-end flow of user creation, authentication, and data retrieval through well-structured, reusable test automation. With integrated logging, reporting, schema validation, and a CI/CD pipeline, the framework reflects a practical and maintainable approach to API test automation.